

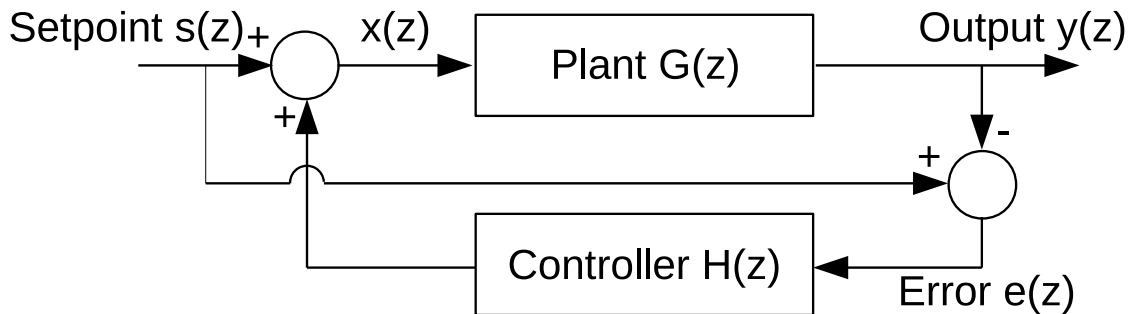
Digital Control Algorithms in an Object-Oriented Framework

1 Description

Control systems are pervasive in Electrical and Mechanical engineering. Digital control is increasingly popular because the required electronic components have become inexpensive and because digital control algorithms are easily modified and can even be made to be adaptive or better handle non-linear systems.

2 Digital Control Systems

We will consider a plant and control system of the following type:



Note that this figure shows the entire system simplified as a digital system. In real industrial systems, the plant will be a continuous-time system (i.e., analogue) and there will therefore be a D/A converter at the plant input and an A/D converter for measuring the plant output.

With digital control, the input to the plant to be controlled is

$$x(z) = s(z) + H(z)e(z) \quad (1)$$

$$= s(z) + H(z)[s(z) - y(z)] \quad (2)$$

where

$s(z)$ = setpoint value

$y(z)$ = plant output

$G(z)$ = plant transfer function

$H(z)$ = controller transfer function

I have given you this as a z-transform domain representation, which is convenient for this type of system. I will explain why I did this in class.

There are numerous types of digital control algorithms. We will look at two of them.

2.1 Proportional Control

With proportional control the error signal is fed back to the input, multiplied by some constant k_c , i.e., $H(z) = k_c$:

$$x(z) = s(z) + k_c e(z)$$

We want to implement this in the discrete time domain. Because this is a very simple system, the discrete time-domain representation is simply

$$x(nT) = s(nT) + k_c e(nT)$$

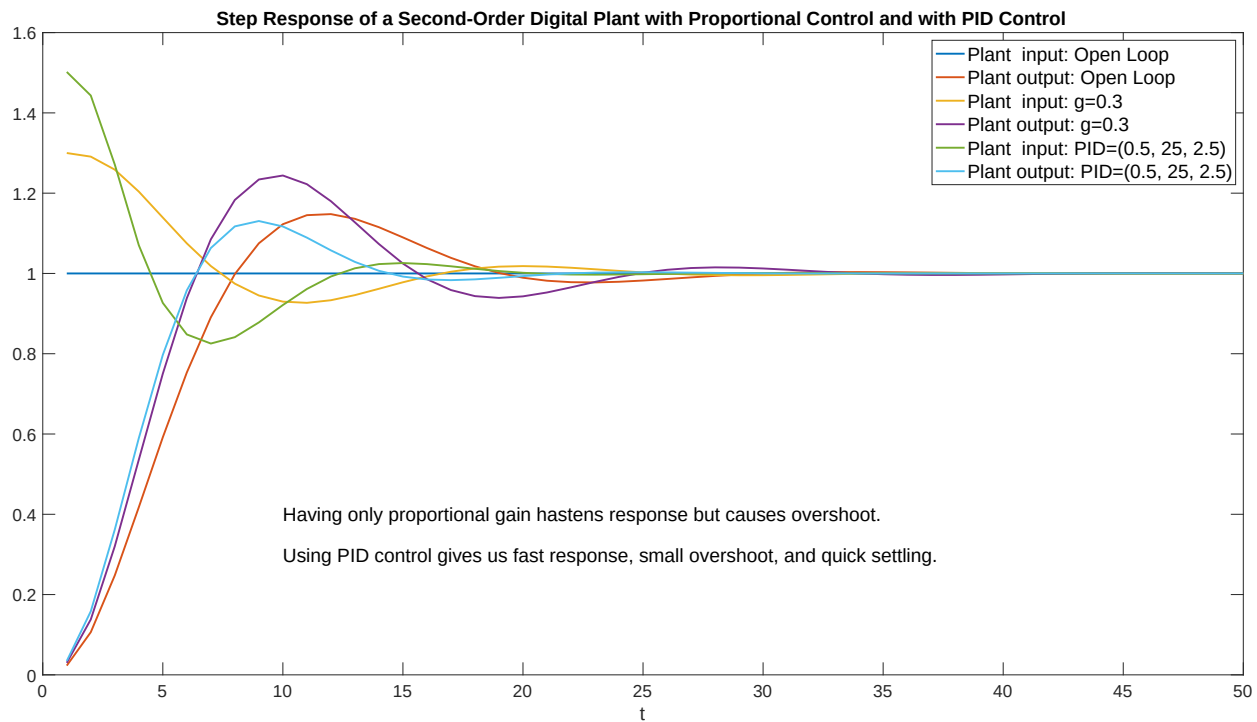
where

$$\begin{aligned} T &= \text{sampling interval} \\ n &= \text{sample number} \end{aligned}$$

When you implement this function, $x(nT)$ means “the current value of x ”. Similarly, $x((n-1)T)$ means “the value of x from the previous time step”. Therefore, *we do not see nT or $(n-1)T$ in our code!* ... but you may need to save the current value of a variable so that you can use it as the “previous” value during the next time step.

I have given you this function as `ProportionalController.cpp` and the class header file `ProportionalController.hpp`.

At this juncture, it is appropriate to look at the following figure:



There are three sets of curves, each showing the plant input $x(n)$ and the plant output $y(n)$:

1. When there is no controller (open loop response).
2. With a proportional controller having a gain $k_c = 0.3$.
3. With a PID controller having a $k_c = 0.5$, $t_I = 25$, and $t_D = 2.5$. More on PID in the next section.

Looking at the outputs for the first two cases.

1. When there is no controller, a step input (i.e., setpoint changing from 0 to 1), the output rises, overshoots the setpoint, and exhibits a damped oscillation as it settle to the setpoint.
2. With proportional feedback, the output more quickly approaches the setpoint but overshoots and oscillates more.

We see that proportional control can accelerate the response, but it is ineffective at reducing overshoot and oscillation.

2.2 Proportional-Integral-Derivative (PID) Control

With PID control the error signal is fed back to the input, but a more complicated calculation is performed. In addition to proportional control, the integral of the error signal is used to reduce the effect of accumulated error and the derivative of the error signal is used for predictive compensation. This algorithm is best shown in the discrete-time domain, rather than in the z-domain (ref. section 11.4 of <http://pc-textbook.mcmaster.ca/Marlin-Ch11.pdf>)

$$x(nT) = s(nT) + k_c \left[e(nT) + \frac{T}{t_I} \sum_{i=1}^n e(i) - \frac{t_D}{T} [y(nT) - y((n-1)T)] \right] \quad (3)$$

or, for those of you who are comfortable with z transforms

$$X(z) = S(z) + k_c \left[E(z) + \frac{T}{t_I} \sum_{i=1}^n E(z) - \frac{t_D}{T} [Y(z) - Y(z^{-1})] \right]$$

where t_I is the integration time constant and t_D is the derivative time constant. For better performance, we will replace the integral portion with a “leaky” approximation to the integral $\frac{T}{t_I} q(nT)$ where

$$\begin{aligned} q(nT) &= 0.9q((n-1)T) + 0.1e(nT) \quad \text{or} \\ Q(z) &= 0.9Q(z^{-1}) + 0.1E(z) \end{aligned}$$

and the variable q is initialized to zero. In the figure above, you can see that the PID controller gives us faster response, less overshoot, and quicker settling time!

You are to write the function `PID_Controller.cpp` and the class header file `PID_Controller.hpp`.

3 Object-Oriented Programming Considerations

I have given you five C++ files:

- `main.cpp` that instantiates a Plant and a Controller and then runs the system for 50 time intervals.
- `Plant.hpp`, `Plant.cpp` defines a linear plant model, i.e., a linear model of a physical plant.
- `Controller.hpp` is an *abstract* class that defines the requirements for a generic controller.
- `ProportionalController.hpp`, `ProportionalController.cpp` is an implementation of a Controller. Specifically, a `ProportionalController` “is-a” `Controller`.

You will notice that in `main.cpp`, I have the following lines:

```
Controller *controller;
ProportionalController propController(0.3);
// Use the following line after you write your code and then
// make controller point to your object.
//PID_Controller pidController(0.5, 25, 2.5);
controller = &propController;
```

N.B. The pointer variable `controller` can point at *anything* that “is-a” Controller! Later, in the `for` loop, `controller` refers to whichever Controller we are using. This is an example of *polymorphism*.

4 Hints

Each `ProportionalController` object needs only to remember its gain (k_c), whereas the `PID_Controller` needs to remember three gains as well as the current value of the integral (q) and the value of the output (y) from the previous control step (so that it can compute the approximation to the derivative). The `ProportionalController` is “stateless” whereas the `PID_Controller` needs to remember its internal state (i.e., it is “stateful”). For the `PID_Controller` to be mathematically “nice”, the values of these two state variables need to be initialized to zero (which, in control systems theory, is needed to have a *linear time-invariant system*).

To initialize multiple variables in the constructor, use a comma-separated list:

```
PID_Controller (const double propGain, const double integralGain) :
    kc(propGain), ti(integralGain) {    }; // More needed!
```

5 A Bit of MATLAB™

The output of the program can be pasted directly into MATLAB to create an array of data Y that can then be plotted using the following commands:

```
figure(1);
plot(Y);
```

6 Assignment

1. Write the code for `PID_Controller.hpp` and `PID_Controller.cpp`.
2. If you have access to MATLAB, plot your step response curve in MATLAB.

7 Requirements

1. Hand in your well-documented `PID_Controller.hpp` and `PID_Controller.cpp`.
2. No need to show me your plot.

BTW, the reason that these pages look marvellous is that the document was typeset using $\text{\LaTeX}$.